Optimizing Pull Request Workflow for PR Prioritization and Reviewer Assignment

Design Project II - SWE 4606

Presented By:

Niaz Rahman

210042159

Antara Arifa Mullick

210042132

Labibah Hoque

210042121

Table of content



Introduction



Literature Analysis



Current Research limitation



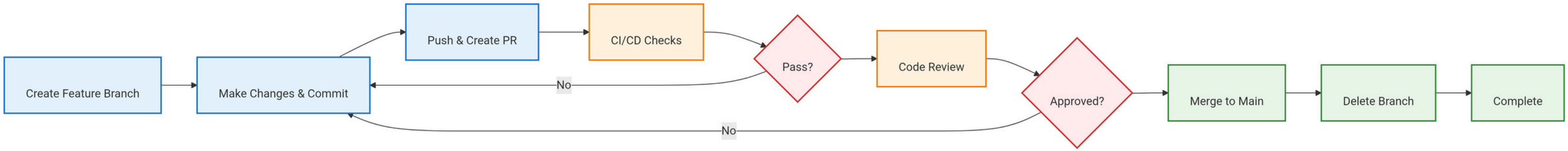
Problem Statement



Future plans

Introduction

Pull Request : a request to merge code changes for review and approval before integration.



Pull Request Lifecycle

Introduction



However the review process isn’t always that fast and efficient which often results in pull request latency

Literature Analysis

“Using Large-scale Heterogeneous Graph Representation Learning for Code Review Recommendations at Microsoft”

Publishing year: 2023

ICSE 2023 - Software Engineering in Practice Track (May 2023)

Paper Link : https://arxiv.org/pdf/2202.02385

Literature Analysis

Problem Addressed:

How to automatically recommend the most suitable code reviewers for new pull requests in large software projects.

Methodology Used:

- The researchers used real Microsoft project data about developers pull requests and file changes
- They built a big graph connecting people files and pull requests
- A Graph Neural Network GNN was trained on this graph to learn who reviews what
- The GNN model then suggested the best reviewers for new pull requests
- The system called CORAL worked better than older reviewer assignment methods

Literature Analysis

“Automated Code Reviewer Recommendation for Pull Requests”

Publishing year: 2024

e-Informatica Software Engineering Journal, Volume 18, Issue 1

Paper Link : https://www.e-informatyka.pl/index.php/einformatica/volumes/volume-2024/issue-1/article-8/

Literature Analysis

Problem Addressed:

In open-source projects, reviews often get delayed because they depend on reviewers noticing and reviewing PRs. Sometimes, the right people don’t see them soon enough, which slows everything down.

Methodology Used:

- **Candidate selection:** All previous reviewers or committers considered; PRs without reviewers excluded.
- **Heuristic approach:** Combines reviewers’ expertise, commit/review metrics, social network features, and recency of activity.
- **Evaluation:** Tested on five open-source GitHub projects.
- **Metrics:** Standard measures—precision@k, recall@k, F1 score—compare predicted vs. actual reviewers.
- **Key distinction:** Focuses on a heuristic algorithm, not machine learning, addressing limitations of prior ML-based methods with narrow features or small datasets.

Literature Analysis

“Improving Merge Pipeline Throughput in Continuous Integration via Pull Request Prioritization”

Paper Publishing year: 2025

IEEE International Conference on Software Maintenance and Evolution (ICSME 2025)

Paper Link : https://www.e-informatyka.pl/index.php/einformatica/volumes/volume-2024/issue-1/article-8/

Literature Analysis

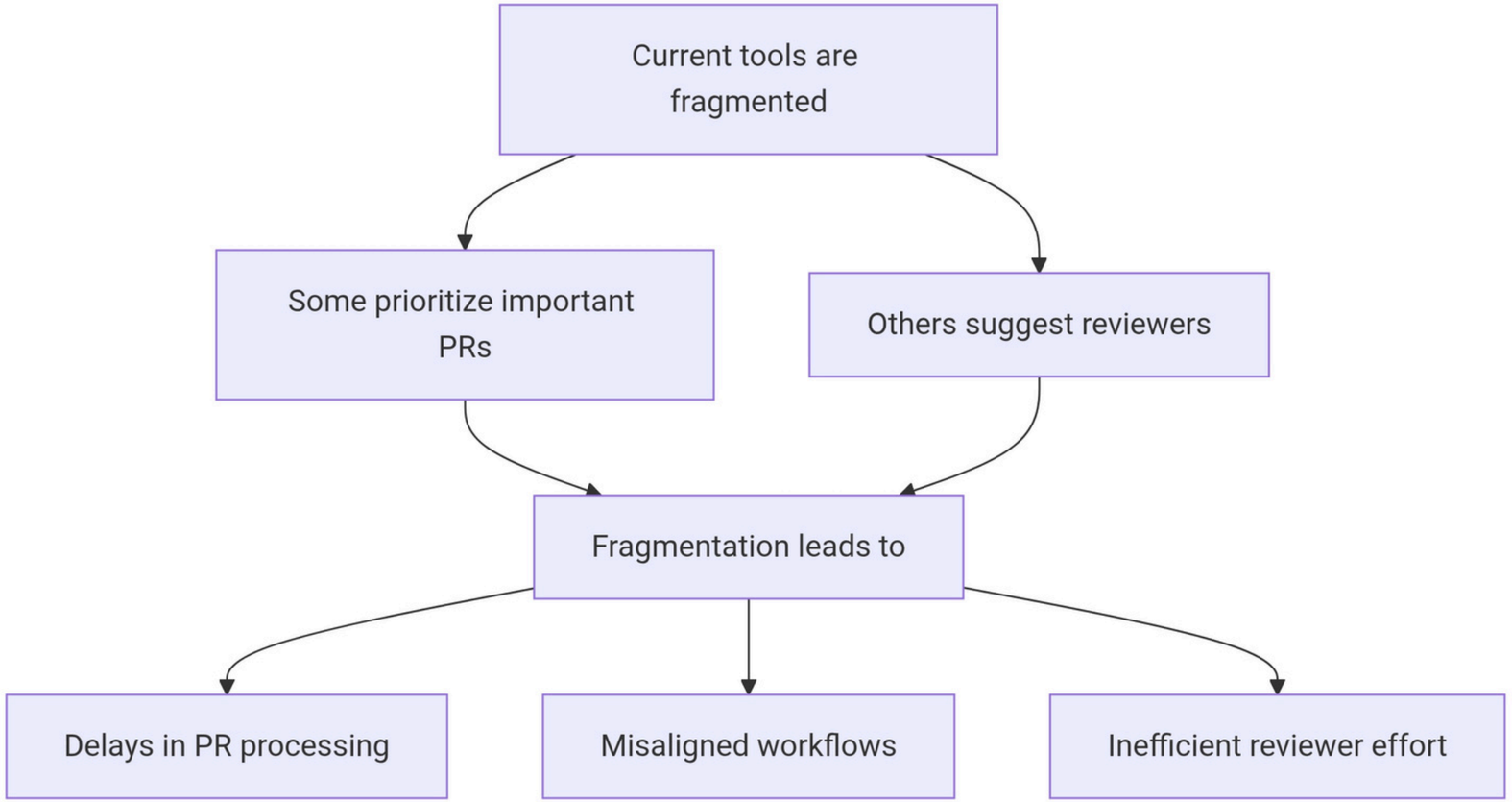
Problem Addressed:

In large projects, failed pull requests can slow down the merge process and waste time. The paper solves this by reordering PRs so that the ones most likely to pass are tested first, speeding up merges and improving efficiency.

Methodology Used:

- The paper builds a machine learning model that predicts which pull requests (PRs) are most likely to fail. Using this prediction, it reorders the merge queue, letting the safer PRs run first and delaying risky ones.
- To do this, it uses data like code changes, number of commits, reviews, and past CI test results. The model (XGBoost) is trained on past PR data and tested to see how well it predicts failures.
- Then, the researchers simulate real merge scenarios and compare their method with normal FIFO ordering and simple rules. Their approach merges more PRs successfully and reduces pipeline delays

Current Research Limitations



Current Research Limitations

Tradiditional ML models has some limitations


Requires large labeled datasets


Needs extensive manual setup


Separate models for each task


Cannot handle complex, context-dependent scenarios


Leads to inflexible systems and slower PR processing

Problem Statement

Addressing the limitations of existing research, we observed that **pull request (PR) prioritization** and **reviewer assignment** are often treated as **separate problems,** despite both being major contributors to **PR latency**. Most current approaches rely on **traditional machine learning techniques**, **heuristics** which, when applied independently, result in **inefficiencies** and fail to provide an **integrated framework** for optimizing both tasks simultaneously.

Conclusion and Future Directions

- Future work will focus on enhancing the refinement of pull request latency using a more integrated system

- We will focus using LLMs to further explore the contextual analysis of pull requests

References

1.DeputyDev -- AI Powered Developer Assistant: Breaking the Code Review Logjam through Contextual AI to Boost Developer Productivity
2.Predicting pull request completion time: a case study on large scale cloud services
3.Improving Merge Pipeline Throughput in Continuous Integration via Pull Request Prioritization
4.Improving Feedback on GitHub Pull Requests: A Bots Approach.
5.An exploratory study of the pull-based software development model
6.TIRR: A Code Reviewer Recommendation Algorithm with Topic Model and Reviewer Influence
7.Pull request latency explained: an empirical overview
8.Mining code review data to understand waiting times between acceptance and merging: an empirical analysis
9.Who Should Review this Pull-Request: Reviewer Recommendation to Expedite Crowd Collaboration
10.Nudge: Accelerating Overdue Pull Requests toward Completion
11.Suggestion Bot: Analyzing the Impact of Automated Suggested Changes on Code Reviews
12.AI-Powered Code Review Assistant for Streamlining Pull Request Merging
13.Using nudges to accelerate code reviews at scale
14.The Impact of Large Language Models (LLMs) on Code Review Process

THANK YOU

For Your Time

